Notes

These notes shall serve as aid for the implementation, and will eventually be integrated into the report;

1 Problem Statement

(P) is a sparse linear system of the form:

$$\begin{bmatrix} D & E^T \\ E & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$$

where $D \in \mathbb{R}^{m \times m}$ is a diagonal positive definite matrix (i.e., $D = \text{diag}(D) > 0$) and $E \in \mathbb{R}^{(n-1) \times m}$ is obtained by removing the last row from the node-arc incidence matrix of a given connected directed graph. These problems arise as the KKT system of the convex quadratic separable Min-Cost Flow Problem, hence you can look, e.g., here for ways to generate meaningful instances of the problem.

- (A1) is GMRES, and you must solve the internal problems $\min \left\| H_n y - \|b\| e_1 \right\|$ by updating the QR factorization of H_n at each step: given the QR factorization of H_{n-1} computed at the previous step, apply one more orthogonal transformation to compute that of H_n.
- (A2)is the same GMRES, but using the so-called *Schur complement preconditioner*

  $$P = \begin{bmatrix} D & 0 \\ E & S \end{bmatrix}$$

  where S is either $S = ED^{-1}E^T$ or a sparse approximation of it (to obtain it, for instance, replace the smallest off-diagonal entries of S with zeros).

No off-the-shelf solvers allowed.

2 GMRES

GMRES is an iterative method for computing an approximation of the solution to the linear system $Ax = y$. It accomplishes such approximation by computing:

$$\min_{x \in K_n(A,y)} \|Ax - y\|$$

i.e. by finding the minimum x within the *Krylov Space* $K_n(A, y)$ such that the norm of the residual $Ax - y$ is minimized.

2.1 Background: Krylov Spaces and Arnoldi

Given a non-necessarily-symmetric matrix (although in our case it is) $A \in \mathbb{R}^{m \times m}$, a vector $y \in \mathbb{R}^m$, and a natural number $n \leq m$:

$$K_n(A, y) = \text{span } (y, Ay, A^2y, \ldots, A^{n-1}y).$$

We can obtain a good basis for this vector space by using the Arnoldi Algorithm:

$$\texttt{Arnoldi}(A, y, n) =: Q_n, \underline{\mathrm{H}}_n$$

Where:

- $Q_n \in \mathbb{R}^{n \times m}$ is the matrix $(q_1 \mid \ldots \mid q_n)$ of vectors in the basis;
- $\underline{\mathrm{H}}_n \in \mathbb{R}^{n+1 \times n}$ is a matrix such that:
  1. $$\underline{\mathrm{H}}_n = \begin{bmatrix} H_n \\ \ldots \end{bmatrix}$$
     where $H_n \in \mathbb{R}^{n \times n}$ is s.t. $H_n = Q_n^T A Q_n$, and can therefore be seen as A in the Krylov subspace.
  2. $$AQ_n = Q_{n+1}\underline{\mathrm{H}}_n \tag{1}$$

Remark 1. *The first vector of the basis generated by* $\texttt{Arnoldi}(A, y, n)$ *is* $\dfrac{y}{\|y\|}$.

2.2 Simplifying the minimum problem

We can simplify the minimum problem to:

$$\min_{z \in \mathbb{R}} \|\underline{\mathrm{H}}_n z - e_1 \|y\|\|$$

Proof.

$$\begin{aligned}
\min_{x \in K_n(A,y)} \|Ax - y\| &= \min_{z \in \mathbb{R}} \|AQ_n z - y\| && (q_i\text{s form basis of } K_n(A,y)) \\
&= \min_{z \in \mathbb{R}} \|Q_{n+1}\underline{\mathrm{H}}_n z - y\| && (\text{by (1)}) \\
&= \min_{z \in \mathbb{R}} \|Q_{n+1}^T(Q_{n+1}\underline{\mathrm{H}}_n z - y)\| && (Q_{n+1}^T \text{ orthogonal}) \\
&= \min_{z \in \mathbb{R}} \|\underline{\mathrm{H}}_n z - Q_{n+1}^T y)\| && (Q_{n+1} \text{ orthogonal}) \\
&= \min_{z \in \mathbb{R}} \|\underline{\mathrm{H}}_n z - Q_{n+1}^T q_1 \|y\|\| && (\text{by Remark 1}) \\
&= \min_{z \in \mathbb{R}} \|\underline{\mathrm{H}}_n z - e_1 \|y\|\| && (Q_{n+1}^T \text{ orthogonal})
\end{aligned}$$

□

2.3 Lanczos

Since the input matrix A is symmetric, we can use the Lanczos Algorithm to iteratively construct the orthonormal basis for the Krylov space. In particular, the H_n matrix resulting from such algorithm will by symmetric and tridiagonal:

$$H_n = \begin{bmatrix} \alpha_1 & \beta_1 & & & & \\ \beta_1 & \alpha_2 & \beta_2 & & & \\ & \beta_2 & \alpha_3 & \ddots & & \\ & & \ddots & \ddots & \beta_{m-2} & \\ & & & \beta_{m-2} & \alpha_{m-1} & \beta_{m-1} \\ & & & & \beta_{m-1} & \alpha_m \end{bmatrix}$$

This allows us to perform the computations without storing the whole matrix, but only the diagonal and subdiagonal. The Julia implementation of the algorithm is as follows: [1]

[1] TODO: Write this as pseudocode

```
function Lanczos(A::SparseMatrixCSC, y::SparseVector, n::Int)

    alpha = zeros(1,n)      # diagonal of H
    beta = zeros(1,n)       # subdiagonal of H (= superdiagonal of H because symmetric
    )

    L = zeros(size(A)[1], n)    # Lanczos Basis

    # base case

    q = y / norm(y)
    L[:, 1] = q
    w = A * q
    alpha[1] = w' * q
    w = w - alpha[1] * q
    beta[1] = norm(w)
    # inductive case

    for j in 2:n
        if j % 500 == 0
            println(j, "-th Lanczos iteration")
        end
        if beta[j-1] < 1e-13
            println("Breakdown");
            return
        end
        q = w / beta[j-1]
        L[:, j] = q

        w = A * q
        alpha[j] = w' * q
        w = w - alpha[j] * q - beta[j-1] * L[:, j-1]
        beta[j] = norm(w)

    end

    return (L, alpha, beta)

end
```

2.4 Lanczos + QR

A trivial way to implement GMRES would be:

- Use Lanczos to obtain a basis for the Krylov space
- Compute the QR factorization of $\underline{\mathrm{H}}_n$
- Use the factorization to easily solve the linear system arising from the minimum problem.

However, the requested version should compute the QR factorization during the Lanczos iteration.

2.4.1 Extending the QR decomposition

Assume we have $\underline{\mathrm{H}}_{n-1} = Q_{old} R_{old}$. $\underline{\mathrm{H}}_n$ will be computed using a Lanczos iteration, obtaining:

$$\underline{\mathrm{H}}_n = \left[\begin{array}{ccc|c} & \underline{\mathrm{H}}_{n-1} & & \begin{matrix}0\\ \vdots \\ 0\\ \beta_{n-1}\\ \alpha_n\end{matrix} \\ \hline 0 & \dots & 0 & \beta_n \end{array}\right]$$

We want to find two matrices Q, R such that:

- $Q \in \mathbb{R}^{(n+1)\times(n+1)}$ is orthogonal
- $R \in \mathbb{R}^{(n+1)\times n}$ is upper triangular
- $\underline{\mathrm{H}}_n = QR$, i.e. $Q^T \underline{\mathrm{H}}_n = R$

First off, to obtain a Q matrix of the requested shape that is orthogonal, we could consider:

$$Q'_{old} = \left[\begin{array}{ccc|c} & Q_{old} & & \begin{matrix}0\\ \vdots \\ 0\end{matrix} \\ \hline 0 & \dots & 0 & 1 \end{array}\right]$$

This takes us close to the solution, but not quite there; in fact the product $Q'^T_{old} \underline{\mathrm{H}}_n$ is *almost* triangular:

$$Q'^T_{old} \underline{\mathrm{H}}_n = \left[\begin{array}{ccc|c} & R_{old} & & Q'^T_{old} \begin{bmatrix}0\\ \vdots \\ 0\\ \beta_{n-1}\\ \alpha_n\end{bmatrix} \\ \hline 0 & \dots & 0 & \beta_n \end{array}\right]$$

Recall that R_{old} is an $n \times (n-1)$ upper triangular matrix, hence it has a row of zeros at the bottom. To make this matrix triangular, we should have a zero in place of the β_n. We can get rid of such zero by performing a Householder reflection on the bottom two elements of the final column. Let these be:

$$x = \begin{bmatrix} a \\ \beta_n \end{bmatrix} \qquad \mapsto \qquad y = \begin{bmatrix} \|x\| \\ 0 \end{bmatrix}$$

As seen in class, the transformation we need is a reflection w.r.t $v = x - y$:

$$H_{refl} = I - \frac{2vv^T}{\|v\|^2}$$

In order for the reflection to only act on the two bottom elements of the final row, we extend it as follows, obtaining another orthogonal transformation:

$$O = \left[\begin{array}{c|cc} & 0 & 0 \\ I & \vdots & \vdots \\ & 0 & 0 \\ \hline 0 \dots 0 & \multicolumn{2}{c}{\multirow{2}{*}{H_{refl}}} \\ 0 \dots 0 & & \end{array}\right]$$

The matrix we obtain as $OQ'^T_{old}\underline{\mathrm{H}}_n$ is triangular.

Constructing the R Matrix

- Get old R;
- Compute:

$$Q'^T_{old}[0, \dots, 0, \beta_{n-1}, \alpha_n]^T = ([0, \dots, 0, \beta_{n-1}, \alpha_n]Q'_{old})^T$$

  Let $q_1 ... q_n$ be the columns of Q'_{old}. ([2]) The result is:

$$\sum_i \beta_{n-1} q_{i,n-1} + \alpha_n q_{i,n}$$

  This can be rewritten, letting $r_1 ... r_n$ be the rows of Q'_{old}, as:

$$\beta_{n-1} r_{n-1} + \alpha_n r_n$$

- Perform the O transformation to obtain R.

[2] Too many Qs: TODO rename Lanczos basis vector to b_i?

Constructing the Q matrix

- Get old Q;
- Construct Q'_{old} by adding the extra row and column;
- Compute $Q = (OQ'^T_{old})^T$.

Altenatively we can calculate the new Q as follows:

$$Q = O^T Q'_{old} = OQ_{old}$$

By the symmetry of O, which is guaranteed by the symmetry of H_{refl}. Computationally, we only need to update the last two columns of Q, so we need not construct O explicitly. The following Julia snippet shows such computation [3]

[3] TODO write pseudocode instead

```
Q[j+1, j+1] = 1.0

newcols = zeros(j+1, 2)

for i in 1:j+1
  # Compute Q_old * O without constructing O
  # Can compute Q_old * O instead of Q_old * O' because O is sym.

  newcols[i, 1]      = Q[i, j] * Hrefl[1,1] + Q[i, j+1] * Hrefl[2,1]
  newcols[i, 2]    = Q[i, j] * Hrefl[1,2] + Q[i, j+1] * Hrefl[2,2]
end

Q[1:j+1, j:j+1] = newcols
```

2.5 Solving the minimum problem, given QR

Given the QR factorization of $\underline{\mathrm{H}}_n$, we can transform our minimum problem:

$$\boxed{\min_{z\in\mathbb{R}} \|\underline{\mathrm{H}}_n z - e_1\|y\|\|} \quad \text{into} \quad \boxed{\min_{z\in\mathbb{R}} \|R_0 z - Q_0^T(e_1\|y\|)\|}.$$

Proof. Let $\underline{\mathrm{H}}_n = QR, \quad Q = [Q_0, Q_c], \quad R = [R_0, 0]^T$.

$$\begin{aligned}
\|\underline{\mathrm{H}}_n z - e_1\|y\|\| &= \|Q^T(\underline{\mathrm{H}}_n z - e_1\|y\|)\| && (Q^T \text{ is norm-preserving}) \\
&= \|Q^T Q R z - Q^T(e_1\|y\|)\| && (\text{QR factorization of } \underline{\mathrm{H}}_n) \\
&= \left\| \begin{bmatrix} R_0 \\ 0 \end{bmatrix} z - \begin{bmatrix} Q_0^T \\ Q_c^T \end{bmatrix} (e_1\|y\|) \right\| && (\text{assumption}) \\
&= \left\| \begin{bmatrix} R_0 z - Q_0^T(e_1\|y\|) \\ -Q_c^T(e_1\|y\|) \end{bmatrix} \right\| && (\text{vector algebra})
\end{aligned}$$

Whose minimum is only influenced by the top portion (the bottom one does not depend on z). □

Once we have transformed the problem to this form, if R_0 is invertible (which is true iff the $\underline{\mathrm{H}}_n$ is full-rank), we have obtained a *triangular system*, which is efficiently solved by backsubstitution.